GraphWorkflow: A Low-Overhead Compile-Once Graph Execution Engine for Multi-Agent Systems

Kye Gomez Adi Chaudhary Ayaan Gazali Steve-Dusty Wyatt Stanke
Swarms
Correspondence: kye@swarms.world

August 2026

Abstract

Multi-agent systems built on large language models (LLMs) are increasingly structured as directed acyclic graphs (DAGs) of agents, in which each node performs one model invocation and each edge carries intermediate results. Because a single agent step costs between 0.5 and 5 seconds of model latency, orchestration frameworks have treated their own overhead as negligible. This assumption fails in three regimes: cold start of short-lived processes, high-frequency re-execution of compiled workflows, and graphs whose width or depth reaches hundreds of nodes. We present GraphWorkflow, the graph orchestration engine of the open-source Swarms framework. GraphWorkflow follows a compile-once, sweep-many design: an explicit compilation phase computes topological generations, materializes adjacency maps in a single pass, validates the graph structurally, and freezes a per-layer execution plan of resolved (node, agent) tuples; the runtime then executes each layer as a wavefront over a single lazily created thread pool, with an inline fast path for single-node layers. A pluggable backend interface allows the same engine to run over NetworkX or over the Rust-based rustworkx library. We develop a cost model that decomposes orchestration overhead into per-node and per-layer terms and use it to attribute measured behavior to individual mechanisms. Using an open benchmark suite of five topologies at 10 to 200 nodes with no-op agents, we compare GraphWorkflow against LangGraph 1.0.4. Across 15 topology and size configurations, GraphWorkflow executes compiled graphs with a geometric-mean speedup of 7.0× (up to 62.5× on 200-node chains), compiles graphs 21.6× to 31.3× faster, and completes the cold build-compile-execute path 7.9× faster. A programming-model comparison shows that the same static-DAG workflow also demands materially less of the user in GraphWorkflow: no state schema, no reducer annotations, no explicit entry and exit wiring, and no recursion-limit configuration. We attribute each measured difference to specific architectural choices, report distributional statistics for every measurement, and release the benchmark harness, analysis scripts, and this document as a reproducible artifact in the Swarms repository at https://github.com/kyegomez/swarms.

1 Introduction

Systems that coordinate multiple LLM agents have converged, largely independently, on the same structural abstraction: a directed graph whose nodes are agent invocations and whose edges carry intermediate outputs forward [13, 16, 22, 23, 31, 42]. The abstraction is attractive for the same reasons dataflow graphs are attractive in distributed computing [12, 17, 25]: dependencies are explicit, independent work is exposed for parallel execution, and the program structure can be validated, visualized, and persisted separately from the code that executes it.

Agent graphs, however, occupy an unusual point in the workload space. A node in an Airflow or Dask graph may run for milliseconds to hours, and the engine amortizes its scheduling cost against cluster-scale resources [3, 32]. A node in an agent graph is a single network-bound call into an LLM serving stack [21], typically 0.5 to 5 seconds, executed in-process, frequently from short-lived scripts, serverless functions, and interactive sessions. In this regime the costs that matter are the cost of constructing the graph, of preparing it for execution, and the residual per-run cost once the graph is warm. Each is invisible in a single request and dominant in aggregate: a service that executes a compiled workflow per incoming request pays the per-run overhead millions of times, a serverless deployment pays the first-run cost on every cold start, and an evaluation harness that sweeps hundreds of workflow variants pays build and compile cost in a loop.

Existing agent-graph runtimes accept substantial overhead in exactly these places. LangGraph, the most widely used graph orchestrator for agents, executes graphs on a Pregel-style superstep loop [24] with channel-based state, reducer functions, and checkpoint hooks consulted on every step (Section 2.3). These mechanisms buy real features, including

cycles, conditional edges, and durable execution, but they are charged to every node of every run, whether or not the workflow uses them. As we show in Section 7, on a 200-node sequential chain of no-op nodes, LangGraph spends 18.15 ms of pure orchestration per execution, roughly 90 μs per superstep, even with checkpointing disabled.

This paper describes GraphWorkflow, the graph execution engine of the open-source Swarms framework, which takes the opposite position: the common case for agent workflows is a static DAG executed one or many times, and a static DAG can be prepared once and executed with almost no per-run machinery. GraphWorkflow separates the graph lifecycle into an explicit compilation phase and a minimal execution phase. Compilation infers entry and exit points, computes topological generations, materializes successor and predecessor maps in one pass over the edge list, runs structural validation from those maps, and freezes a per-layer execution plan in which every node is already resolved to a (node id, agent object, node type, display name) tuple. Execution is then a sweep over the frozen plan: each layer is dispatched as a wavefront onto a single thread pool that is created lazily, sized to the widest layer, and shared across the entire run; a layer containing a single node bypasses the pool entirely and runs inline. Nothing on the hot path traverses the graph, consults a channel, or acquires a checkpoint.

A second architectural decision makes the engine independent of any one graph library. All structural operations go through a small backend interface with two implementations: one over NetworkX [15] and one over the Rust-based rustworkx [38], which maps string node identifiers to integer indices and lowers generation computation and reachability into native code. The backend affects only build and compile cost; the frozen execution plan makes the runtime backend-agnostic.

We evaluate GraphWorkflow with an open benchmark harness that measures build, compile, first-run, and steady-run cost across five topologies at 10 to 200 nodes, with no-op agents so that every measured microsecond is framework overhead. Against LangGraph 1.0.4 we find geometric-mean steady-state speedups of 7.0× to 7.2×, rising to 62.5× on deep chains where per-superstep overhead compounds; compile-time speedups of 21.6× to 31.3×; and cold-path speedups of 7.9×. The performance gap has a usability counterpart: expressing the same static DAG requires materially fewer concepts in GraphWorkflow than in LangGraph, a comparison we make concrete with side-by-side implementations of the same workflow (Section 4.2). We also quantify the feature asymmetry between the two systems and isolate the share of LangGraph overhead attributable to its state reducer with an ablation.

This paper makes four contributions. The first is a cost taxonomy for agent orchestration (Sections 2 and 3): four measurable lifecycle phases and a per-node plus per-layer overhead model that turns benchmark topologies into probes of individual mechanisms. The second is the design of the engine itself (Section 4): the backend abstraction, a compilation pipeline with single-pass adjacency materialization and validation by multi-source reachability, the frozen execution plan, and a layer-wavefront scheduler with a shared lazy thread pool and an inline single-node fast path, together with per-layer checkpointing, nested subgraphs, multi-loop refinement, and a programming-model comparison showing the API obligations each framework imposes on the static-DAG common case. The third is a formal account of the execution model (Section 3), establishing that layered execution respects dependency order, that compilation is $O(V+E)$, and that the layer-barrier runtime connects to classical work-span scheduling bounds. The fourth is a reproducible evaluation (Section 7): the harness, raw samples, analysis scripts, and figures are released, results are medians of 9 samples with 95% confidence intervals reported alongside their variance, and each measured gap is attributed to a specific architectural cause.

Section 2 develops the workload argument and analyzes the architecture of existing orchestrators. Section 3 formalizes the execution model. Sections 4 and 5 describe design and implementation. Sections 6 and 7 present the methodology and results. Sections 8 and 9 survey related systems and limitations, and Section 10 concludes.

2 Background and Motivation

2.1 Agent graphs

A multi-agent workflow assigns a role-specialized agent to each step of a larger task; the natural representation is a DAG in which each node wraps one agent and each edge declares that the downstream agent consumes the upstream agent's output. The pattern now spans the field: single-agent reasoning loops [43] compose into pipelines for software teams [16, 31], role-play societies [23], simulated populations of interacting agents [29], and ensembles that aggregate multiple models [40]. Frameworks differ in how the graph is expressed (node and edge objects, flow strings, decorator DSLs, conversation policies) but converge on the same runtime obligation: execute every node after its predecessors, pass predecessor outputs forward, and run independent nodes concurrently.

Two properties distinguish this workload from classic dataflow. First, node execution is dominated by network-bound LLM inference, so thread-based concurrency is sufficient even under a global interpreter lock, which is released during I/O. Second, graphs are small by distributed-systems standards (tens to hundreds of nodes) but are created and executed

frequently: per request in services, per cold start in serverless deployments, per configuration in evaluation sweeps. Framework overhead is thus paid at high frequency against modest per-graph work, precisely the regime in which constant factors and startup costs dominate asymptotics.

2.2 A cost taxonomy for orchestration

We separate the life of a workflow into four measurable phases, which structure both the design discussion and the evaluation. *Build* constructs the in-memory graph, adding N nodes and M edges. *Compile* covers whatever preprocessing the framework performs between construction and first execution: validation, topological analysis, plan generation. *First run* is the cold path, build plus compile plus one execution; it is what a fresh process pays before producing its first result, and its operational weight is familiar from serverless platforms, where cold-start latency is a first-order concern [33, 41]. *Steady run* is one execution of an already-prepared graph, the marginal cost a long-lived service pays per request. Reporting both ends keeps the comparison honest across deployment regimes: a one-shot script cares about the cold path, a long-lived service or evaluation sweep about the steady state.

2.3 The architecture of LangGraph

LangGraph [22] executes a `StateGraph` on a runtime explicitly modeled on Pregel [24] and the bulk synchronous parallel model [39]. Nodes communicate through named *channels*; execution proceeds in *supersteps*. In each superstep the runtime determines the active nodes, invokes them, collects their writes as pending updates, and applies those updates to the channels through *reducer* functions that define how concurrent writes merge (append to a list, add to a counter, overwrite). Between supersteps the runtime evaluates conditional edges, checks interrupt conditions, and offers the updated state to an optional checkpointer that can persist every superstep for durable execution, in the style of workflow platforms such as Temporal [37], and for human-in-the-loop resumption. Graph construction is followed by an explicit `compile()` that assembles this channel and superstep machinery.

This architecture is a rational design for its target semantics. Cycles and conditional control flow require re-evaluating which nodes are active at every step; durable execution requires a serialization point between steps; concurrent writes to shared state require a merge discipline. The cost is that the machinery executes per superstep and per channel write regardless of whether the graph uses any of it. A static DAG with pure dataflow edges, the common case for pipeline-style agent workflows, still pays for active-node computation, channel updates, reducer invocation, and checkpoint hooks on every step. Three consequences are measurable in Section 7: per-superstep cost of roughly 90 to 120 μs that scales with graph depth (Figure 10); compile cost that grows to 12.57 ms at 200 nodes as the channel graph is assembled (Figure 6); and state that accumulates across supersteps when the reducer appends, which we isolate with a reducer ablation (Section 7.6). The comparison in this paper should therefore be read as a measurement of what generality costs on the static-DAG common case, not as a claim that the generality is worthless.

2.4 Why cluster DAG engines do not fill the gap

Mature DAG execution exists in adjacent ecosystems, none matching the in-process agent regime. Workflow managers such as Airflow [3], Luigi [35], Prefect [30], and Metaflow [28] schedule tasks through a database-backed control plane at per-task latencies of hundreds of milliseconds to seconds, three orders of magnitude above the budget of an in-process agent step. The dataflow lineage running from Dryad [17] and Spark [44] through Naiad [27] and TensorFlow [1] optimizes the throughput of data-parallel operators on clusters, and its Python-native descendants, Dask [32], Parsl [4], and Ray [25], bring per-task overhead down to roughly 100 microseconds to 1 millisecond, but all of them assume a running scheduler or cluster runtime and serialize arguments across process boundaries, and both assumptions are wrong for a library call inside a request handler passing Python strings between agents. Graph libraries such as NetworkX [15], igraph [10], and rustworkx [38] provide the algorithms but no execution semantics. The niche GraphWorkflow targets is the composition: library-grade latency, dataflow semantics, and graph-library algorithms behind a single in-process object.

2.5 Design requirements

The preceding analysis yields five requirements that drive Section 4: (R1) per-run overhead must not scale with anything but the work actually dispatched, so no per-run graph traversal or state machinery; (R2) all analysis that can be hoisted out of the run loop must be, behind an explicit, cached, invalidation-safe compile step; (R3) concurrency must exploit I/O-bound parallelism without paying thread-pool construction per layer; (R4) the engine must not be married to one

graph library, since algorithmic backends improve independently of orchestration; and (R5) reliability features (validation, checkpointing, callbacks) must be pay-for-use, costing nothing when disabled.

3 Theoretical Foundations

This section fixes notation, defines the layered execution semantics that GraphWorkflow implements, establishes its correctness and complexity, and derives the cost model used in Section 7.

3.1 Graphs, layers, and schedules

Definition 1 (Workflow graph)**.** A workflow graph is a finite directed graph $G = (V, E)$ with $E \subseteq V \times V$, together with an assignment $a : V \to \mathcal{A}$ of an executable agent to each node. We write $N = |V|$ and $M = |E|$. For $v \in V$, $\mathrm{pred}(v) = \{u : (u, v) \in E\}$ and $\mathrm{succ}(v) = \{w : (v, w) \in E\}$. Entry points are nodes with $\mathrm{pred}(v) = \emptyset$; exit points are nodes with $\mathrm{succ}(v) = \emptyset$.

Definition 2 (DAG and depth)**.** G is a DAG if it admits no directed cycle. For a DAG, the depth of a node is the length of the longest directed path from any entry point to it,

$$d(v) = \begin{cases} 0 & \mathrm{pred}(v) = \emptyset, \\ 1 + \max_{u \in \mathrm{pred}(v)} d(u) & \text{otherwise}, \end{cases} \tag{1}$$

and the depth of the graph is $d(G) = 1 + \max_v d(v)$, the number of distinct depth values.

Definition 3 (Topological generations)**.** The i-th generation (equivalently, *layer*) of a DAG is $L_i = \{v \in V : d(v) = i\}$ for $0 \le i < d(G)$. The generations partition V, and every edge $(u, v) \in E$ satisfies $d(u) < d(v)$.

Generations are the canonical maximal-parallelism layering: L_0 is exactly the set of nodes executable immediately, and inductively L_i is exactly the set of nodes executable once $L_0, \ldots, L_{i-1}$ have completed. Any schedule that starts every node no earlier than its generation index preserves dependencies; any layering with more layers than $d(G)$ forfeits parallelism.

Definition 4 (Layered wavefront schedule)**.** Given generations $L_0, \ldots, L_{d-1}$ and a pool of p workers, the layered wavefront schedule executes layers in index order; within a layer, all nodes are submitted to the pool and the schedule advances only when every node of the layer has completed (a per-layer barrier). Node v receives as input the task τ and the outputs $\{y_u : u \in \mathrm{pred}(v)\}$, and produces $y_v = a(v)\big(\pi(\tau, \{y_u\})\big)$ for a prompt-assembly function π.

Proposition 1 (Dependency safety)**.** *Under the layered wavefront schedule, every node begins execution only after all of its predecessors have completed, and the value y_u read for each predecessor is the final value produced by u in the current loop iteration.*

Proof. Let $(u, v) \in E$. By the generation property $d(u) < d(v)$, so u's layer precedes v's layer. The per-layer barrier guarantees that layer $d(u)$ has fully completed, and its outputs have been recorded into the output map, before any node of layer $d(v)$ is submitted. The output map is written only by the recording step that follows each node's completion and is read only during prompt assembly for later layers, so no read can observe a partial or stale value. Within a layer no node reads another's output, since edges within a layer would contradict $d(u) < d(v)$. □

Remark 1. The barrier is stricter than necessary: an edge-triggered scheduler may start v as soon as $\mathrm{pred}(v)$ completes, even if unrelated nodes of earlier layers are still running. Section 3.3 quantifies the gap, and Section 9 discusses why GraphWorkflow accepts it.

3.2 Computing generations

GraphWorkflow computes generations with Kahn's algorithm [9, 19] (Algorithm 1), either natively in the backend (rustworkx lowers the loop into Rust) or in Python as a fallback that additionally tolerates cycles by emitting the cyclic remainder as a final layer so that validation, not execution, is the component that reports the error. The queue-based formulation is chosen over the depth-first alternative [36] because it yields the generations directly rather than a single total order that would have to be re-partitioned into layers.

Algorithm 1 Layering by Kahn's algorithm with cyclic remainder

Require: directed graph $G = (V, E)$
Ensure: layers $L_0, L_1, \ldots$; unplaced nodes appended as final layer

```
indeg[v] ← |pred(v)| for all v ∈ V
F ← [ v ∈ V : indeg[v] = 0 ];   L ← [ ];   placed ← 0
while F ≠ ∅ do
    append F to L;   placed ← placed + |F|
    F' ← [ ]
    for all v ∈ F do
        for all w ∈ succ(v) do
            indeg[w] ← indeg[w] − 1
            if indeg[w] = 0 then append w to F'
            end if
        end for
    end for
    F ← F'
end while
if placed < |V| then
    append {v : indeg[v] > 0} to L          ▷ cyclic remainder, surfaced by validation
end if
return L
```

Theorem 1 (Compilation complexity)**.** *For a workflow graph with N nodes and M edges, GraphWorkflow compilation (entry and exit inference, generation computation, adjacency materialization, structural validation, and execution-plan freezing) runs in $O(N + M)$ time and $O(N + M)$ space.*

Proof. Entry and exit inference scan the degree of each node, $O(N)$ given degree access, and $O(N + M)$ if degrees are derived from the edge list. Algorithm 1 decrements each edge exactly once and enqueues each node exactly once, $O(N + M)$. Adjacency materialization is a single pass over the edge list into preallocated per-node lists, $O(N + M)$. Validation (Section 4.3) consists of degree checks, one acyclicity test by the same Kahn argument, and two multi-source breadth-first traversals, each of which visits every node and edge at most once, $O(N + M)$. Plan freezing iterates the layered node list once, $O(N)$. All auxiliary structures (in-degree map, frontier lists, adjacency maps, plan tuples) are linear in $N + M$. □

3.3 Work, span, and the layer-barrier cost model

Let t_v be the execution time of node v, $T_1 = \sum_v t_v$ the total work, and T_∞ the weight of the heaviest directed path (the critical path, or span). Greedy schedulers on p workers satisfy the classical bound $T_p \le T_1/p + T_\infty$ [6, 7, 14]; makespan-optimal scheduling under precedence constraints is hard in general, with even the two-processor case requiring specialized algorithms [8], which is part of why practical runtimes settle for greedy or barrier disciplines. The layer-barrier schedule instead achieves

$$T_p^{\text{barrier}} \;=\; \sum_{i=0}^{d-1} \Big(\max_{v \in L_i} t_v \;+\; \beta_i \Big) \qquad \text{when } p \ge \max_i |L_i|, \tag{2}$$

where β_i is the framework's per-layer dispatch and collection overhead. For unit-time nodes, $\sum_i \max_{v \in L_i} t_v = d(G) = T_\infty$, so the barrier schedule is span-optimal whenever layers are internally balanced; it loses to edge-triggered scheduling only when node durations within a layer are skewed and successors of the fast nodes could have started early. Wide layers do inherit the tail-latency amplification familiar from distributed request fan-out, where the slowest of k parallel calls sets the layer's latency [11]. For LLM agents, whose latencies cluster within a band determined by the model and output length, the skew term is nonetheless small relative to node duration, which is the empirical justification for choosing the simpler barrier semantics.

The quantity this paper measures is not t_v (zero for no-op nodes) but the overhead $\sum_i \beta_i$ plus any per-node dispatch cost. We model steady-run orchestration overhead as

$$\Omega(N, d) \;\approx\; \alpha_{\text{node}} \cdot N \;+\; \alpha_{\text{layer}} \cdot d \;+\; \alpha_0, \tag{3}$$

with α_{node} the marginal cost of dispatching and recording one node, α_{layer} the marginal cost of one barrier (or superstep), and α_0 fixed per-run cost. The topology suite of Section 6 is designed to separate the two coefficients: the *chain* ($d = N$, one

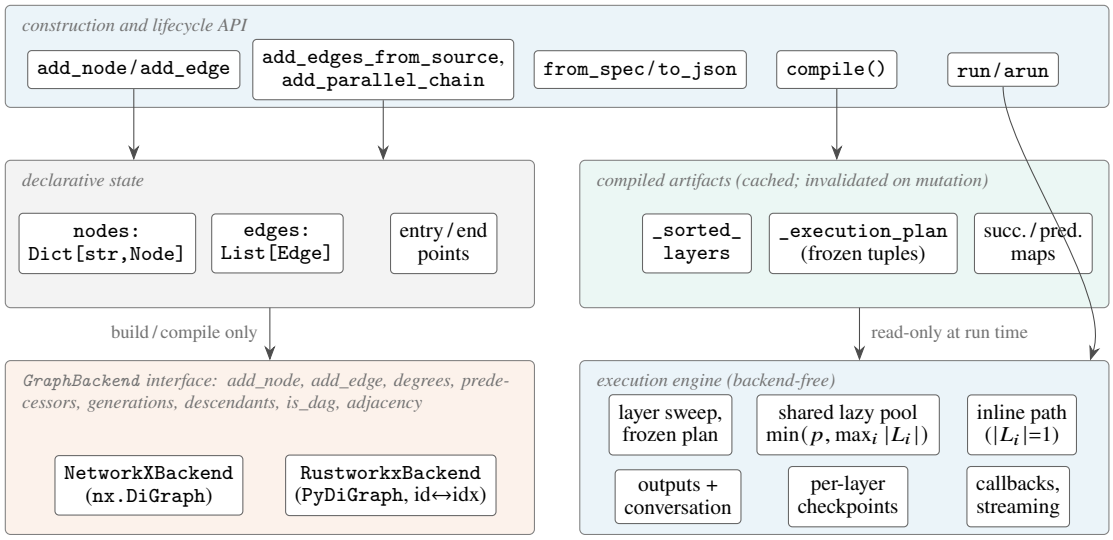


Figure 1: GraphWorkflow architecture. Construction mutates declarative state and the backend graph; `compile()` distills both into cached artifacts; the run engine touches only the frozen execution plan and the predecessor map, never the backend.

node per layer) loads both terms equally, while *wide* ($d = 2$) isolates α_{node}. Section 7.4 fits observed medians to this form and shows that GraphWorkflow and LangGraph differ primarily in α_{node} and α_{layer} by one to two orders of magnitude, and moreover that GraphWorkflow's inline fast path drives α_{node} below the threaded dispatch cost on single-node layers.

Over n executions in one process, cumulative orchestration cost is first-run cost plus $(n-1)$ times steady-run cost; since GraphWorkflow leads in both terms across every measured configuration (Section 7), the comparison is regime-independent, and the steady-run coefficient is the one that dominates for any long-lived deployment.

4 The Design of GraphWorkflow

GraphWorkflow is a single Python class in the Swarms framework [13] that owns a node table, an edge list, a graph backend, compiled artifacts, and an execution engine. Figure 1 shows the structure; this section walks through it top-down.

4.1 Nodes, edges, and construction

A `Node` binds a string identifier to an executable and a type tag: `AGENT` for a Swarms agent (any object exposing `run(task, ...)`) or `SUBGRAPH` for a nested GraphWorkflow instance, which enables hierarchical composition (Section 4.7). Identifiers default to the agent's name, and constructors (`Node.from_agent`, `Edge.from_nodes`) accept agents, nodes, or raw strings interchangeably, so edges can be declared directly between agent objects. An `Edge` is an ordered pair of identifiers with optional metadata; it declares pure dataflow, with no conditions, guards, or reducers attached.

Beyond single-edge insertion, bulk constructors express common shapes in one call and one invalidation: `add_edges_from_source` (fan-out), `add_edges_to_target` (fan-in), and `add_parallel_chain` (complete bipartite stage). A declarative `from_spec` path builds a workflow from node and edge lists, and `to_json` round-trips the full structure including compiled-state metadata.

Every mutator ends by invalidating the compiled artifacts, so a stale plan can never execute; the next `compile()` or `run()` rebuilds them. This makes the compile step idempotent and safe to call eagerly (the constructor auto-compiles when it receives a complete graph).

4.2 The programming model in practice

The design choices above surface directly in the code a user writes. Figure 2 implements the same diamond workflow, one analyst fanning out to a writer and a researcher whose outputs an editor merges, in both frameworks, using each framework's native API for exactly this static-DAG shape.

Three differences are structural rather than cosmetic, and each traces to a design decision from Section 2.3. First, GraphWorkflow has no user-visible state: agents are nodes, edges are dataflow, and the engine assembles each node's prompt from its predecessors' outputs (Section 4.5), so per-node results come back keyed by node identifier. LangGraph

(a) GraphWorkflow

```python
from swarms import Agent, GraphWorkflow

analyst = Agent(agent_name="Analyst",
                model_name="gpt-5.4")
writer = Agent(agent_name="Writer",
                model_name="gpt-5.4")
researcher = Agent(agent_name="Researcher",
                model_name="gpt-5.4")
editor = Agent(agent_name="Editor",
                model_name="gpt-5.4")

wf = GraphWorkflow()
for a in (analyst, writer,
          researcher, editor):
    wf.add_node(a)

wf.add_edge(analyst, writer)
wf.add_edge(analyst, researcher)
wf.add_edge(writer, editor)
wf.add_edge(researcher, editor)

task = "Produce a market report."
results = wf.run(task)
# {"Analyst": ..., "Writer": ...,
# "Researcher": ..., "Editor": ...}
```

(b) LangGraph

```python
import operator
from typing import Annotated, TypedDict
from langgraph.graph import (
    StateGraph, START, END)

class State(TypedDict):
    # fan-in needs a reducer to merge
    # concurrent writes
    results: Annotated[list, operator.add]

def as_node(agent):
    # wrap each agent as a state function
    def node(state: State):
        out = agent.run(str(state["results"]))
        return {"results": [out]}
    return node

builder = StateGraph(State)
for name, ag in [("analyst", analyst),
                 ("writer", writer),
                 ("researcher", researcher),
                 ("editor", editor)]:
    builder.add_node(name, as_node(ag))

builder.add_edge(START, "analyst")
builder.add_edge("analyst", "writer")
builder.add_edge("analyst", "researcher")
builder.add_edge("writer", "editor")
builder.add_edge("researcher", "editor")
builder.add_edge("editor", END)

graph = builder.compile()
task = "Produce a market report."
out = graph.invoke(
    {"results": [task]},
    {"recursion_limit": 100})
# outputs interleaved in one shared list
```

Figure 2: The same diamond workflow in both frameworks. GraphWorkflow treats agents as nodes and infers everything else; LangGraph requires a typed state schema, a reducer for the fan-in, wrapper functions, explicit `START`/`END` wiring, an explicit compile, and a recursion-limit override for deep graphs.

routes all inter-node communication through the typed channel state, so the user must design a schema, choose a reducer for every key that concurrent branches write (omitting it raises a concurrent-update error on the diamond's fan-in), and decide inside each node function what to read from the accumulated state. Second, graph boundary and readiness are inferred in GraphWorkflow (entry and exit points from degrees, compilation on first run), while LangGraph requires them to be declared (`START` and `END` edges, an explicit `compile()`). Third, scale changes the code in LangGraph but not in GraphWorkflow: any graph deeper than the default 25 supersteps needs a per-invocation `recursion_limit` override (the benchmark chains of Section 7 require it at $N = 50$ and above), whereas GraphWorkflow's sweep has no depth configuration at any benchmarked size. Table 1 summarizes the obligations. The comparison is scoped to the static-DAG common case: workflows that need conditional edges or cycles cannot be expressed in GraphWorkflow at all (Section 9), and there LangGraph's obligations buy capability rather than ceremony.

4.3 Compilation

Compilation (Algorithm 2) hoists everything the run loop would otherwise repeat, and four decisions determine its cost profile. The first is a single adjacency pass. Validation and prompt assembly both need neighbor sets, and rather than issuing per-node backend calls (N Python round trips, each crossing the backend abstraction), `compile()` materializes successor and predecessor maps in one iteration over the edge list and derives a per-node tuple cache for predecessors, so that at run time a predecessor lookup is a dictionary hit returning an immutable tuple. The second is structural validation driven from those maps rather than from the backend: the checks confirm that every node has an executable, flag isolated nodes and missing edges, test acyclicity in $O(N{+}M)$ (rather than enumerating simple cycles, which is exponential in their count [18]), and verify reachability in both directions, every node reachable from some entry point and co-reachable from some exit point, with both reachability checks running as multi-source breadth-first traversals seeded with the whole entry or exit set, one $O(N{+}M)$ traversal each instead of one traversal per source. Compile-time validation reports rather than raises, keeping construction ergonomic, and a strict `validate(raise_on_error=True)` is available for CI-style gating.

Table 1: API obligations for expressing a static DAG of agents. Each row is a concept the user must supply; obligations marked required in LangGraph exist because its runtime is general enough for cycles and conditional control flow (Section 2.3).

Obligation	GraphWorkflow	LangGraph
Node definition	agent object itself	function over typed state
Shared-state schema	none	`TypedDict` required
Fan-in merge policy	automatic (predecessor map)	reducer annotation required
Predecessor context	assembled by the engine	read from state by user code
Entry / exit points	inferred from degrees	explicit `START`/`END` edges
Compilation	implicit on first run	explicit `compile()`
Deep graphs ($d > 25$)	no configuration	raise `recursion_limit`
Result form	dict keyed by node	shared channels, merged by reducer

Algorithm 2 GraphWorkflow compilation

Require: nodes V, edges E, backend B; optional entry/exit sets
Ensure: frozen plan P, layers $\mathcal{L}$, adjacency maps, validity report
if compiled flag set **then return** ▷ cached; mutation clears the flag
end if
if entry points unset **then** entry $\leftarrow \{v : \text{indeg}(v) = 0\}$
if exit points unset **then** exit $\leftarrow \{v : \text{outdeg}(v) = 0\}$
$\mathcal{L} \leftarrow B$.topological_generations() ▷ Algorithm 1, native in rustworkx
(succ, pred) $\leftarrow B$.adjacency() ▷ one pass over the edge list
predCache$[v] \leftarrow$ tuple(pred$[v]$) for all v
(errors, warnings) $\leftarrow$ FastValidate$(V, E,$ succ, pred) ▷ Algorithm 4, appendix
$P \leftarrow \left[\left[(v,\ a(v),\ \text{type}(v),\ \text{name}(v)) \ :\ v \in L_i \right] \ :\ L_i \in \mathcal{L} \right]$
set compiled flag; record timestamp

The third decision is the frozen execution plan itself: the compiler resolves, per layer, each node identifier to a 4-tuple of identifier, agent object, node type, and display name, so the run loop performs no dictionary lookups into the node table, no `getattr` resolution, and no type dispatch beyond a tag comparison; the plan is a list of lists of tuples, contiguous, immutable, and cheap to iterate. The fourth is caching with explicit invalidation: the compiled flag makes `compile()` a no-op when nothing changed, which is what makes multi-loop execution (Section 4.8) and repeated `run()` calls pay compilation once. Figure 3 shows the resulting lifecycle.

4.4 The scheduler

Execution (Algorithm 3) is deliberately unsophisticated: it is a double loop over a precomputed list, and the performance-relevant decisions are in what it does *not* do. The run loop never touches the backend; layer membership, ordering, and predecessor sets were all fixed at compile time, so the marginal work per node is prompt assembly (string concatenation over predecessor outputs), one callable dispatch, and two dictionary stores. Thread management follows the same discipline. Pool construction and teardown dominated early profiles of layered execution, because a pool per layer costs a thread spawn and join once per layer, which on a 200-layer chain is 200 pool lifecycles per run; GraphWorkflow therefore creates at most one pool per `run()` call, on first use, sized $\min(p, \max_i |L_i|)$ where p defaults to 95% of the CPU core count (configurable via `max_parallel_nodes`), and shuts it down once in a `finally` block, so purely sequential graphs never create a pool at all. A layer with one node gains nothing from a pool either, since submitting to a worker thread adds a queue handoff, a context switch, and a future wait to a call the scheduler must anyway block on, so singleton layers execute on the calling thread. On chain topologies this eliminates the pool entirely, which is why GraphWorkflow's chain overhead per node ($\approx 1.5\,\mu$s, Section 7.4) is an order of magnitude below its own threaded per-node cost ($\approx 10\,\mu$s) and two orders below LangGraph's per-superstep cost.

Parallel layers collect futures in completion order, so a slow node does not delay the recording, callback, and streaming of its siblings. A node that raises is recorded as a tagged error string; the layer completes, downstream prompts see the failure text, and the run continues, matching the agent-workflow reality that one flaky model call should degrade, not abort, a fan-out of dozens. Observability hooks are pay-for-use: per-node completion callbacks and per-token streaming callbacks are closed over only when registered, and the common path binds a plain `agent.run` call with no keyword-argument copying. Figure 4 illustrates the resulting schedule on a diamond graph.

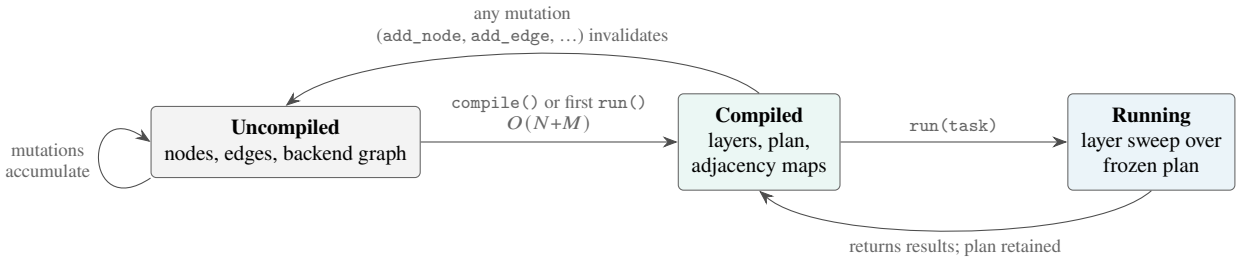


Figure 3: Graph lifecycle. Compilation artifacts are cached across runs and loops; every structural mutation returns the workflow to the uncompiled state, so a stale plan can never execute.

Algorithm 3 Layered wavefront execution (single loop shown)

Require: frozen plan P, task τ, predecessor cache, workers p
Ensure: output map R

$W \leftarrow \max_i |P_i|$; $X \leftarrow \varnothing$ ▷ pool created lazily
$R \leftarrow \{\}$; prev $\leftarrow \{\}$
for $i = 0, 1, \ldots, |P| - 1$ **do**
  if checkpointing enabled **and** layer i has a saved result **then**
    restore into R, prev; **continue**
  end if
  $D \leftarrow [\ (v, a, ty, nm,\ \pi(v, \tau, \text{prev}))\ :\ (v, a, ty, nm) \in P_i\ ]$ ▷ prompts built up front
  if $|D| = 1$ **then**
    execute the single call inline on the caller's thread ▷ no pool handoff
  else
    if $X = \varnothing$ **then** $X \leftarrow \text{ThreadPool}(\min(p, W))$
    end if
    submit every call in D to X
    for all futures as they complete **do**
      collect output (or capture the error as the node's output)
    end for
  end if
  for all completed (v, nm, ty, y_v) **do**
    prev$[v] \leftarrow y_v$; $R[v] \leftarrow y_v$; append (nm, y_v) to conversation
    fire `on_node_complete`(v, y_v) if registered
  end for
  persist layer outputs if checkpointing enabled
end for
return R ▷ pool shut down once, after all loops

4.5 State, prompts, and the conversation log

GraphWorkflow deliberately has no channel system. The run's state is one dictionary mapping node identifiers to outputs. Prompt assembly for node v reads only pred(v) from the compiled tuple cache and concatenates their outputs under an instruction template; entry nodes receive the raw task. Because the output map is append-only and layer barriers order all cross-layer accesses (Proposition 1), no locking is needed. A structured `Conversation` object accumulates the same outputs for observability, an $O(1)$ append per node.

4.6 Fault tolerance: per-layer checkpointing

When a checkpoint directory is configured, the engine persists each layer's outputs to a JSON file keyed by a SHA-256 digest of the task and the layer index (Python's salted `hash()` is unstable across processes). On re-execution of the same task, saved layers are restored, replayed into the conversation for state equivalence, and skipped. The granularity mirrors the execution model: the barrier is already a consistency point, so a layer snapshot is atomic with respect to the schedule and the maximum lost work after a crash is one layer. Checkpointing costs nothing when disabled, consistent with R5.

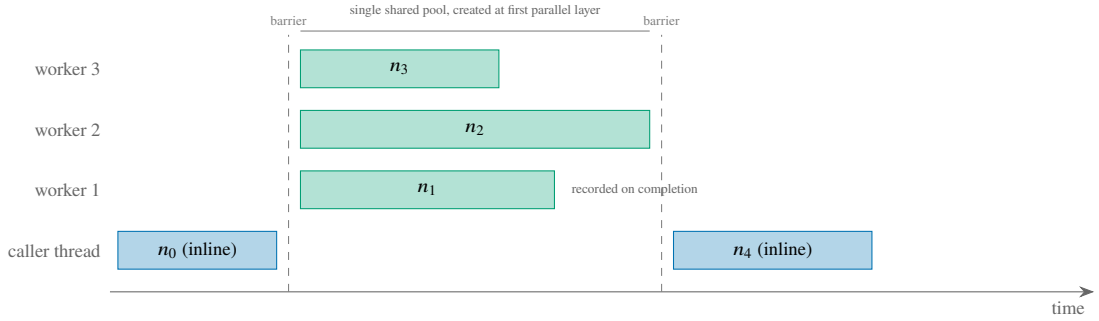


Figure 4: Wavefront schedule of a diamond graph ($n_0 \to \{n_1, n_2, n_3\} \to n_4$). Singleton layers run inline on the caller's thread; the pool is created once, at the first layer that needs it, and outputs are recorded as futures complete.

4.7 Hierarchical composition

A node may wrap an entire GraphWorkflow instance. At dispatch, the child receives the assembled prompt as its task and runs its own compiled plan, inheriting a namespaced checkpoint subdirectory when the parent checkpoints; its result map is flattened into a readable transcript before recording. This gives teams-of-teams composition with no change to the scheduler: the child is just a long-running node.

4.8 Iterative refinement

With `max_loops` > 1, the engine re-sweeps the compiled plan, feeding the previous iteration's exit-point outputs to the next iteration's entry nodes as refinement context, in the spirit of self-refinement loops for language agents [34]. The plan and the pool are reused across loops and per-loop results are retained under loop-suffixed keys, turning one static plan into a fixed-point iteration scheme at no orchestration cost per loop beyond the sweep itself.

5 Implementation

GraphWorkflow is implemented in roughly 4,000 lines of Python inside `swarms/structs/graph_workflow.py`, with no mandatory dependencies beyond NetworkX; rustworkx and Graphviz are optional and detected at import. This section records the implementation details that the evaluation later holds responsible for measured behavior.

5.1 The backend abstraction

`GraphBackend` defines eleven operations, the complete structural vocabulary of compilation and validation: node and edge insertion, degrees, predecessor iteration, reversal, topological generations, simple cycles, descendants, an $O(N{+}M)$ acyclicity predicate, and single-pass adjacency export. Two properties keep the abstraction cheap. First, it is exercised only during build, compile, and validation, so backend dispatch cost cannot appear in steady-run measurements (Section 7.3 confirms the backends are statistically indistinguishable there). Second, the interface is algorithm-level rather than primitive-level: generations and reachability are single calls, letting a native backend keep the whole loop out of Python.

The NetworkX backend is a thin wrapper over `nx.DiGraph` that delegates generations to `nx.topological_generations` and implements `adjacency()` as one pass over `G.edges`. The rustworkx backend has more to do, because rustworkx addresses nodes by integer index: it maintains bidirectional maps between workflow string identifiers and indices, adding nodes idempotently through a get-or-create path. Generations use `rx.topological_generations` natively in Rust, falling back to a Python Kahn implementation (Algorithm 1) that layers cyclic graphs instead of raising, so that validation remains the component that reports cycles; descendant queries call `rx.descendants` rather than a hand-rolled BFS (the replaced implementation also used `list.pop(0)`, an $O(n)$ dequeue); and cycle checks short-circuit through `rx.is_directed_acyclic_graph` before ever enumerating simple cycles, since enumeration is exponential in the number of cycles [18]. The interface is small enough that further backends, igraph [10] being the obvious candidate, would need only the same eleven operations.

5.2 Cache and invalidation discipline

All compiled artifacts live in five private fields (compiled flag, layers, execution plan, successor map, predecessor tuple cache) plus a timestamp. A single `_invalidate_compilation()` clears all of them and is called from every mutator: node and edge insertion, bulk constructors, entry and exit setters, and deserialization paths. The invariant is one-directional: artifacts exist only if they describe the current graph. There is no partial recompilation; at the measured compile costs (0.44 ms for 200 nodes, Section 7.1), incremental compilation would optimize a non-bottleneck.

5.3 Concurrency mechanics

The pool is a `ThreadPoolExecutor` subclass that propagates context (telemetry spans) into workers. Threads suffice because agent nodes are I/O-bound: CPython releases the global interpreter lock during network waits [5], so a layer of k LLM calls overlaps to wall-clock max rather than sum. The no-op benchmark of Section 7 is the adversarial case for this design, all dispatch and no I/O, and Section 7.4 shows the resulting threaded per-node cost ($\approx 10\,\mu$s) still undercuts LangGraph's superstep machinery by an order of magnitude. Worker count defaults to $\lfloor 0.95 \cdot \text{cores} \rfloor$, clamped to at least 1 and capped by the widest layer so narrow graphs never over-allocate threads.

5.4 Serialization and tooling

Workflows serialize to JSON in both directions: `to_json()` captures nodes, edges, entry and exit points, and runtime configuration; `from_spec()` and `from_topology_spec()` construct workflows declaratively (the benchmark harness uses this path). Visualization emits Graphviz renderings with a text fallback; `validate()` re-runs structural validation on demand and can raise for CI-style gating; `get_compilation_status()` exposes cache state, layer count, and worker sizing. None of these paths touch the run loop.

5.5 Memory behavior

The engine's steady-state allocations are intentionally flat: the frozen plan and adjacency caches are built once per compilation; per run, the allocator sees one output dictionary, one prompt string per node, one conversation entry per node, and transient futures for parallel layers. No graph copies are made on the run path (the reversed graph needed by legacy validation was eliminated in favor of the predecessor map), and no per-layer executors or queues are constructed. We did not instrument allocation-level memory profiles for this paper; the benchmark suite currently records wall-clock phases only (Section 6.4), and we flag peak-RSS and allocation-count measurement as future benchmark work rather than asserting unmeasured numbers.

6 Experimental Methodology

6.1 Harness

All measurements come from an open harness (`tests/benchmarks/graph_workflow_benchmarks/` in the Swarms repository) that measures the four phases of Section 2.2 for three implementations: GraphWorkflow over NetworkX, GraphWorkflow over rustworkx, and LangGraph's `StateGraph`. Every node is a no-op function, so measured time is orchestration overhead by construction; there is no LLM latency anywhere in the data. Each cell runs two discarded warmup iterations followed by 9 timed samples; medians are the headline statistic, and means, standard deviations, 95% confidence intervals (Student's t, 8 degrees of freedom), and all raw samples are preserved in the released JSON. The garbage collector is forced between samples and disabled inside each timed region, so a collection triggered by one sample is never billed to a later one, and logging is silenced, since swarms installs an enqueueing loguru sink whose pickle-and-queue cost would otherwise land inside timed regions. The repository root is pinned onto `sys.path` so that the numbers describe the checked-out code rather than a stale installed package, and every result file records platform, core count, Python and library versions, git revision, and dirty-tree status. All results in this paper were produced on macOS-15.7.7-arm64-arm-64bit (12 cores), Python 3.12.3, swarms 14.0.0 at revision `daa3368f` (clean tree), langgraph 1.0.4, networkx 3.5, rustworkx 0.17.1.

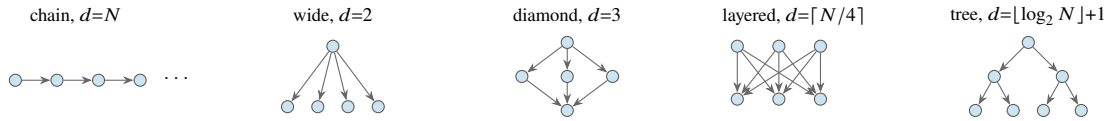


Figure 5: The five benchmark topologies. Chain and layered are the discriminating cases for per-layer overhead; wide isolates per-node dispatch; diamond is the common map-reduce shape; tree gives logarithmic depth.

6.2 Topology suite

Five topologies at $N \in \{10, 50, 200\}$ nodes are chosen to separate the coefficients of the overhead model in Equation (3); Figure 5 sketches them. *Chain* maximizes depth ($d = N$): it stresses per-layer cost and is the discriminating case for superstep-style runtimes. *Wide* is one root fanning out to $N - 1$ leaves ($d = 2$): it isolates per-node dispatch cost and stresses the concurrency path. *Diamond* is fan-out to $N - 2$ workers and fan-in ($d = 3$), the map-reduce shape most common in agent practice. *Layered* stacks fully connected stages of four ($d = \lceil N/4 \rceil$, densest edge count): depth and width together. *Tree* is a binary tree ($d = \lfloor \log_2 N \rfloor + 1$): logarithmic depth with geometric width growth.

6.3 Fairness controls

Both frameworks execute identical topologies, node for node and edge for edge, through their native construction APIs. LangGraph's default 25-superstep recursion cap is raised so deep graphs complete rather than error. Fan-in nodes in LangGraph require a state reducer for concurrent writes; the headline runs use an appending list reducer, which matches the information GraphWorkflow inherently retains (predecessor output map and conversation log). Because an appending reducer grows state across supersteps, we additionally measured a full sweep with an $O(1)$ counter reducer and report the difference as an ablation (Section 7.6), so no headline claim rests on reducer choice. No checkpointer is configured for LangGraph (its default), and GraphWorkflow runs single-loop with checkpointing disabled.

6.4 Threats to validity and missing measurements

The suite measures framework overhead, not end-to-end agent performance; with no-op nodes there is no I/O to overlap, so it exercises dispatch cost, not concurrency benefit. It covers the synchronous path only (neither `arun` nor `ainvoke`), a single machine and Python version, and graphs up to 200 nodes. The two systems are not feature-equivalent: LangGraph's superstep cost purchases conditional edges, cycles, interrupts, and durable execution that GraphWorkflow's sweep does not provide, so results quantify the price of that generality on static DAGs rather than a like-for-like race (Section 9). The harness does not measure memory footprint or CPU utilization; we therefore report no memory or CPU numbers anywhere in this paper and mark them as future benchmark extensions.

7 Evaluation

We organize the evaluation around five questions: how build and compile scale with graph size (RQ1); what the cold path costs (RQ2) and what the steady state costs (RQ3); which architectural mechanisms explain the differences (RQ4); and whether the results are robust to LangGraph's state-reducer choice (RQ5). Unless noted, numbers are medians of 9 samples; error bars and ± values are 95% confidence intervals. Table 2 and Table 4 contain the complete grids.

7.1 RQ1: Build and compile

Build cost (Table 4, Appendix A) is near-linear for all three implementations and differs by a modest constant factor (2.4× geometric mean in GraphWorkflow's favor): inserting nodes and edges is cheap everywhere, and LangGraph's builder does somewhat more bookkeeping per insertion.

Compilation (Figure 6) separates the architectures. Both systems expose an explicit compile step, but they build different things. GraphWorkflow compilation is the $O(N{+}M)$ pipeline of Algorithm 2: generations, one adjacency pass, reachability validation, plan freezing. LangGraph compilation assembles the Pregel runtime: channels per state key and node trigger, write entries, and branch machinery. At 200 nodes, GraphWorkflow compiles the chain in 0.44 ms (networkx) and 0.26 ms (rustworkx) against 12.57 ms for LangGraph: a 21.6× to 31.3× geometric-mean gap that reaches 60.7×

Table 2: First-run (build + compile + one execution) and steady-run medians with 95% confidence intervals, in milliseconds, for all topologies and sizes. All nodes are no-ops; every value is pure orchestration overhead.

		GraphWorkflow (networkx)		GraphWorkflow (rustworkx)		LangGraph	
Topology	N	first (ms)	steady (ms)	first (ms)	steady (ms)	first (ms)	steady (ms)
chain	10	0.37 ± 0.15	0.06 ± 0.01	0.28 ± 0.03	0.06 ± 0.01	2.22 ± 0.16	1.19 ± 0.12
	50	0.49 ± 0.12	0.12 ± 0.01	0.41 ± 0.03	0.11 ± 0.01	8.69 ± 0.35	3.46 ± 0.06
	200	1.23 ± 0.03	0.29 ± 0.01	1.06 ± 0.10	0.32 ± 0.02	36.01 ± 1.07	18.15 ± 4.14
wide	10	0.66 ± 0.03	0.46 ± 0.03	0.61 ± 0.03	0.45 ± 0.08	2.64 ± 0.28	1.27 ± 0.09
	50	1.41 ± 0.05	1.05 ± 0.06	1.34 ± 0.04	1.04 ± 0.02	8.69 ± 0.18	4.31 ± 0.10
	200	3.00 ± 0.09	2.10 ± 0.23	2.68 ± 0.06	1.95 ± 0.05	40.90 ± 3.79	24.51 ± 6.91
diamond	10	0.72 ± 0.05	0.46 ± 0.06	0.60 ± 0.03	0.41 ± 0.01	2.61 ± 0.10	1.31 ± 0.08
	50	1.42 ± 0.03	1.01 ± 0.05	1.38 ± 0.09	1.04 ± 0.06	9.98 ± 1.16	4.38 ± 0.16
	200	3.30 ± 0.09	2.04 ± 0.02	2.89 ± 0.17	1.99 ± 0.05	41.75 ± 0.73	23.23 ± 0.13
layered	10	0.67 ± 0.03	0.46 ± 0.02	0.66 ± 0.03	0.45 ± 0.02	2.89 ± 0.10	1.34 ± 0.03
	50	1.50 ± 0.56	0.97 ± 0.05	1.39 ± 0.03	0.94 ± 0.07	11.19 ± 0.29	4.63 ± 0.15
	200	4.40 ± 0.14	2.75 ± 0.09	4.26 ± 0.32	2.95 ± 0.11	44.41 ± 1.42	19.07 ± 0.11
tree	10	0.62 ± 0.03	0.50 ± 1.29	0.65 ± 0.07	0.48 ± 0.39	2.57 ± 0.10	1.41 ± 0.03
	50	1.48 ± 0.06	1.11 ± 0.03	1.40 ± 0.04	1.15 ± 0.04	8.74 ± 0.46	4.43 ± 0.42
	200	3.45 ± 2.46	2.92 ± 2.50	2.93 ± 0.09	2.08 ± 0.05	35.57 ± 5.75	17.19 ± 0.41

Table 3: Speedup of GraphWorkflow over LangGraph (ratio of medians, LangGraph/GraphWorkflow) aggregated over all 15 topology-size cells.

	vs. networkx backend			vs. rustworkx backend		
Phase	geo. mean	min	max	geo. mean	min	max
Build	2.4×	1.8×	3.7×	2.7×	1.9×	4.4×
Compile	21.6×	11.5×	35.7×	31.3×	14.2×	60.7×
First run	7.9×	3.6×	29.3×	8.7×	3.9×	33.9×
Steady-state run	7.0×	2.7×	62.5×	7.2×	2.8×	55.9×

at size 200 (Table 3). The rustworkx backend's advantage is confined to exactly this phase, as predicted by the architecture: generations and traversals run in native code, roughly halving compile time at 200 nodes, while steady-run cost is untouched (Section 7.3).

7.2 RQ2: The cold path

First-run cost (Figure 7) compounds the build, compile, and one-execution gaps: geometric-mean speedups of 7.9× (networkx) and 8.7× (rustworkx), growing with size to 29.3× on the 200-node chain, where LangGraph pays both its compile cost and 200 supersteps of execution. On the layered topology at 200 nodes, the heaviest cell in the suite, the totals are 4.40 ms versus 44.4 ms. For interactive construction (notebooks, request-scoped workflow assembly), this is the phase users feel.

7.3 RQ3: Steady-state execution

Steady-state execution (Figures 8 and 9) is the regime a long-lived service pays per request. Three observations.

First, the GraphWorkflow backends are indistinguishable (their confidence intervals overlap in every cell), which is the designed consequence of the frozen execution plan: after compile, no backend code executes, so the choice of graph library cannot affect run cost. This is a useful falsifiable check on the implementation's own claim, and the data passes it.

Second, the gap to LangGraph is topology-dependent in exactly the direction the architectural analysis predicts. On shallow graphs (wide, diamond at 10 to 50 nodes) LangGraph pays few supersteps and the ratio sits at 2.7 to 4×. As depth

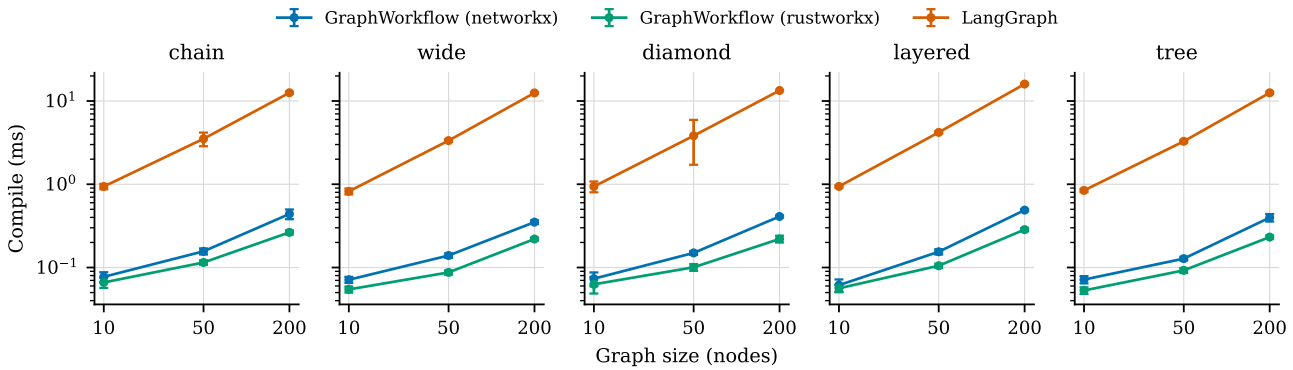


Figure 6: Compile cost versus graph size, log-log. At 200 nodes LangGraph compilation costs 12.5 to 16.0 ms against 0.26 to 0.49 ms for GraphWorkflow; the rustworkx backend roughly halves GraphWorkflow compile time again by computing generations natively.

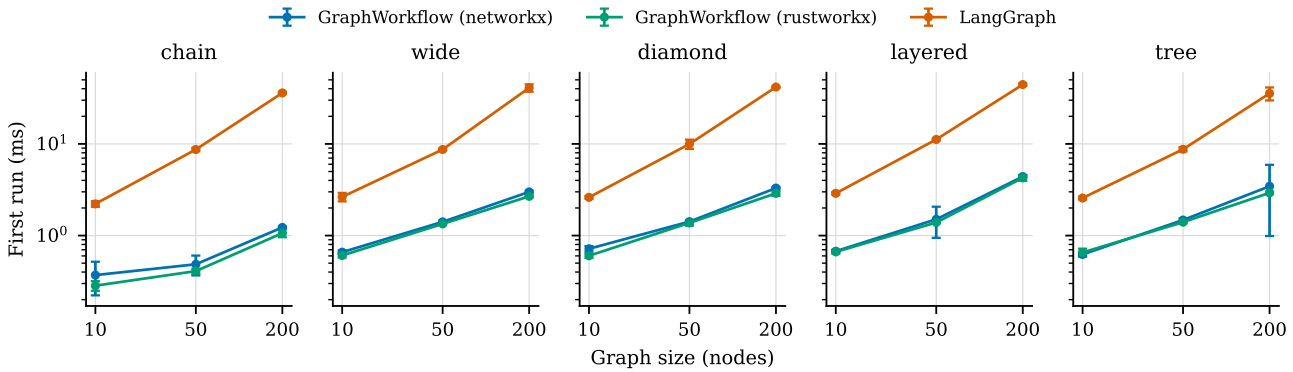


Figure 7: First-run cost (build + compile + one execution from scratch) versus graph size, log-log.

grows the superstep loop compounds: on the 200-node chain GraphWorkflow executes in 0.29 ms versus 18.15 ms, a 62.5× difference. The geometric mean across the grid is 7.0×.

Third, absolute numbers matter for interpretation: at 200 nodes GraphWorkflow's overhead is 0.3 to 3 ms per run, LangGraph's 17 to 25 ms. Against agent work of 0.5 to 5 s per node, neither is visible in a single small workflow; the difference matters at request rates (an orchestration tax of 18 ms per request is 1.8% of a 1 s budget and compounds under fan-out), in evaluation sweeps, and in cold-start-sensitive deployments, where at 1,000 executions of the 200-node chain the cumulative orchestration totals are roughly 0.3 s for GraphWorkflow against 18.2 s for LangGraph.

7.4 RQ4: Attribution

The topology suite doubles as a set of probes for the overhead model of Equation (3). The chain, where every layer is a singleton, prices GraphWorkflow's inline path at 0.29 ms/200 $\approx$ 1.5 μs per node, essentially the cost of assembling a short prompt and two dictionary stores. The wide topology, where 199 nodes dispatch through the shared pool, prices threaded dispatch at 2.10 ms/199 $\approx$ 10.5 μs per node: the pool handoff costs roughly 9 μs, which quantifies exactly what the singleton fast path saves. For LangGraph, the same probes give 90.7 μs per superstep on the chain and 122.5 μs per node on the wide graph; a one-coefficient per-node model fits its whole grid with $R^2 = 0.97$ at $\alpha_{\text{node}} \approx 107\,\mu$s, consistent with overhead that is dominated by per-node channel reads, task preparation, write application, and reducer invocation rather than by the barrier itself. No such single-coefficient model fits GraphWorkflow (α_{node} differs by 7× between the inline and threaded paths), which is the measurable signature of the singleton fast path. Figure 10 shows both signatures directly.

Summing up the attribution: the compile-time gap is LangGraph channel-graph assembly versus one Kahn pass plus one adjacency pass (Figure 11); the steady-run gap at depth is 107 μs versus 1.5 μs per sequential node, the direct price of superstep machinery on a static chain; the steady-run gap at width is 122 μs versus 10.5 μs per parallel node; and the backend choice matters only where backend code actually runs, at compile time.

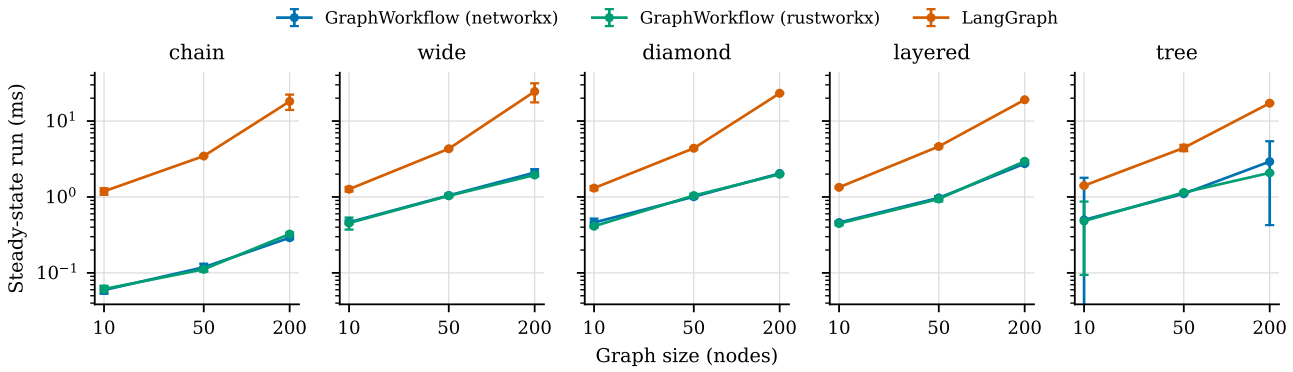


Figure 8: Steady-state execution of a compiled graph versus size, log-log, with 95% confidence intervals. The two GraphWorkflow backends coincide, confirming the run path is backend-free; the gap to LangGraph widens with depth.

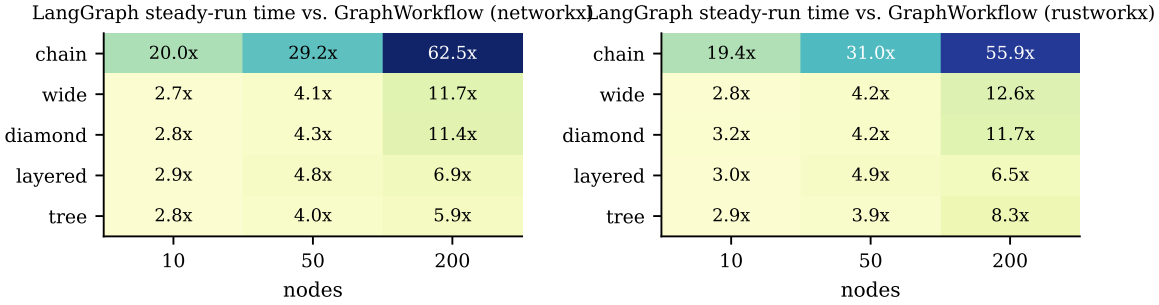


Figure 9: Steady-run speedup (LangGraph median over GraphWorkflow median) across the full grid. The gradient tracks depth: shallow wide graphs sit at 2.7 to 4×, the 200-node chain at 62×.

7.5 Measurement variance

Figure 12 shows complete distributions for the heaviest sequential cell. Both systems are stable: the median coefficient of variation across all steady-run cells is 4.3% for LangGraph and 6.9% for GraphWorkflow-networkx. Relative variance is, if anything, lower for LangGraph, because its per-cell medians are 6 to 60× larger while absolute jitter (thread scheduling, allocator noise) is similar for both; none of the reported ratios is within noise of parity, with non-overlapping confidence intervals in every cell of Table 2.

7.6 RQ5: Reducer ablation

A natural objection to the headline comparison is that the appending list reducer inflates LangGraph's numbers by accumulating state across supersteps. Figure 13 tests this directly with a full sweep under an $O(1)$ counter reducer. The geometric-mean ratio of list to counter steady-run medians is 1.05×; most cells move by less than 12%, with a single small-graph outlier (chain at 10 nodes) at 1.53×. Even scoring LangGraph with its counter numbers everywhere, the 200-node chain remains at 16.8 ms versus 0.29 ms, a 58× gap. The overhead is orchestration, not state accumulation.

8 Related Work

The closest system, and this paper's primary comparison, is LangGraph [22], a Pregel-style channel runtime purpose-built for cyclic, stateful, resumable agent graphs (Section 2.3). The broader agent-framework landscape spans several coordination styles built over the same single-agent substrate of tool-using reasoning loops [43]. AutoGen [42] structures multi-agent work as conversations whose control flow emerges from message policies, which is expressive for open-ended collaboration but offers no static structure to compile or validate. CrewAI [26] organizes agents into role-based crews with sequential or hierarchical process templates rather than arbitrary DAGs. MetaGPT [16] and ChatDev [31] encode software-team standard operating procedures as agent pipelines, CAMEL [23] studies two-agent role-play societies, and generative-agent simulations [29] scale interacting agents into populated environments; all of these express dependency structures that a general DAG engine can host. Reflexion [34] motivates the iterative-refinement loop that GraphWorkflow exposes as `max_loops`. Within Swarms itself [13], GraphWorkflow is the general-graph member of a family of coordination

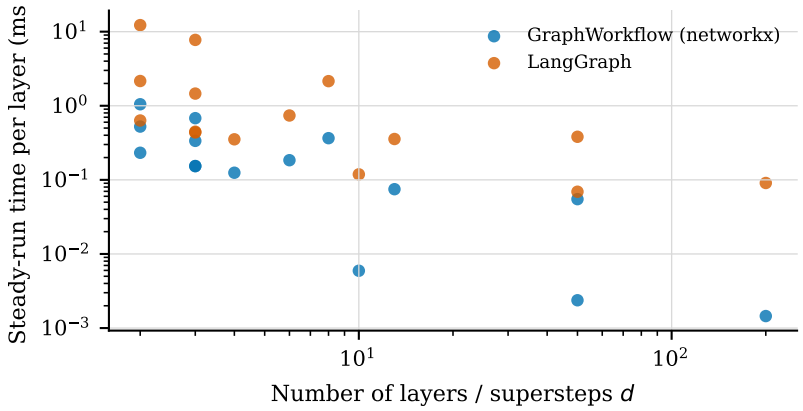


Figure 10: Steady-run cost divided by layer count, against layer count. LangGraph's per-superstep cost is flat at roughly 90 to 600 μs; GraphWorkflow's per-layer cost falls with depth because deep graphs take the inline path.

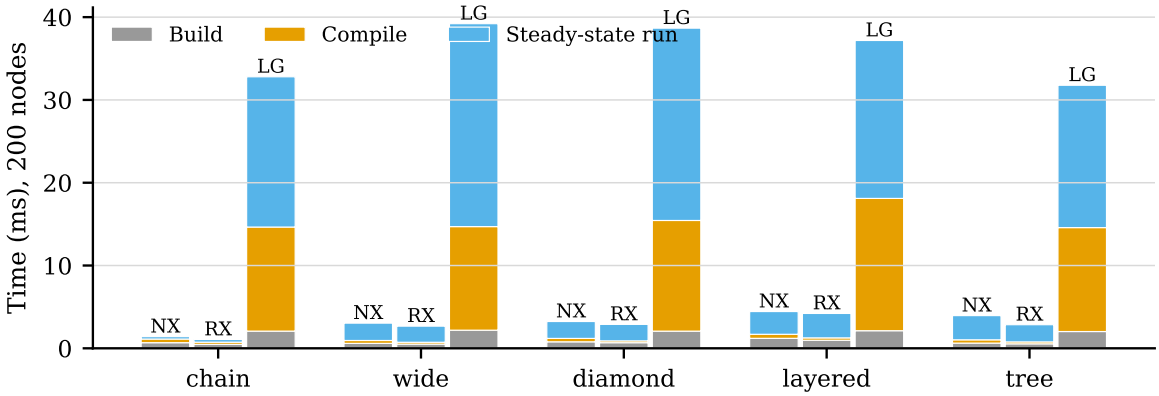


Figure 11: Where the milliseconds go at 200 nodes: build + compile + one steady run, stacked, for GraphWorkflow-networkx (NX), GraphWorkflow-rustworkx (RX), and LangGraph (LG). Compilation dominates LangGraph's preparation cost; execution dominates its recurring cost.

structures (sequential and concurrent workflows, hierarchical swarms, mixture-of-agents [40]), all special cases expressible as graphs.

A second body of related work is workflow and dataflow infrastructure. Workflow managers, including Airflow [3], Luigi [35], Prefect [30], and Metaflow [28], schedule DAGs of tasks across workers with persistent metadata stores, retries, and calendars, and durable execution platforms such as Temporal [37] provide the crash-consistent replay semantics that LangGraph's checkpointing approximates in-process. Their unit economics (per-task latencies of 10^2 to 10^3 ms against GraphWorkflow's 1.5 to 10.5 μs dispatch) reflect a different contract: durable cross-machine state and operational tooling for long-running jobs, not in-process request-path execution. The dataflow lineage is older and faster: Dryad [17] generalized MapReduce [12] to arbitrary DAGs of operators, Spark [44] added in-memory lineage, Naiad [27] added timely iteration, and TensorFlow [1] compiled dataflow graphs for machine-learning workloads. Python-native descendants bring these ideas in-language: Dask [32] executes dynamic task graphs at roughly 100 to 200 μs per task, Parsl [4] targets scientific workflows, and Ray [25] compiles task DAGs for repeated execution over a distributed object store. All target data-parallel computation across processes and machines, serializing arguments at boundaries. Pregel [24] and the BSP model [39] introduced the superstep-and-barrier execution style that LangGraph adapts; GraphWorkflow's layer sweep is itself a degenerate BSP schedule with the communication phase reduced to dictionary writes. The scheduling theory underlying Section 3.3 descends from Graham, Brent, Coffman and Graham, and Blumofe and Leiserson [6–8, 14], and the tail-latency behavior of wide layers is the classic fan-out effect described by Dean and Barroso [11].

Finally, graph libraries supply the algorithmic substrate. NetworkX [15], igraph [10], and rustworkx [38] implement the traversals and orderings; Kahn's algorithm [19], depth-first alternatives [9, 36], and Johnson's cycle enumeration [18] are the classical results the implementation leans on and, in the last case, deliberately avoids on the common path. GraphWorkflow's contribution relative to these libraries is the backend abstraction that makes them interchangeable beneath one execution engine, plus the compile-once discipline that keeps them off the hot path entirely.

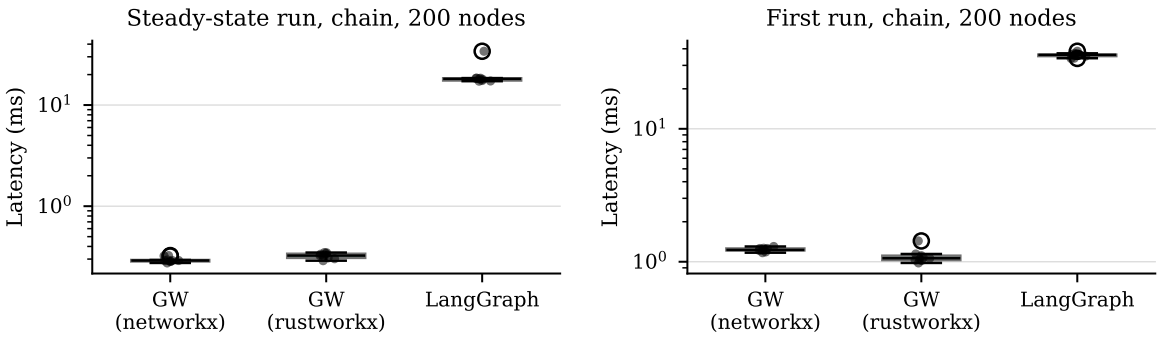


Figure 12: Full sample distributions (9 samples, log scale) for the 200-node chain. Boxes span quartiles; dots are individual samples.

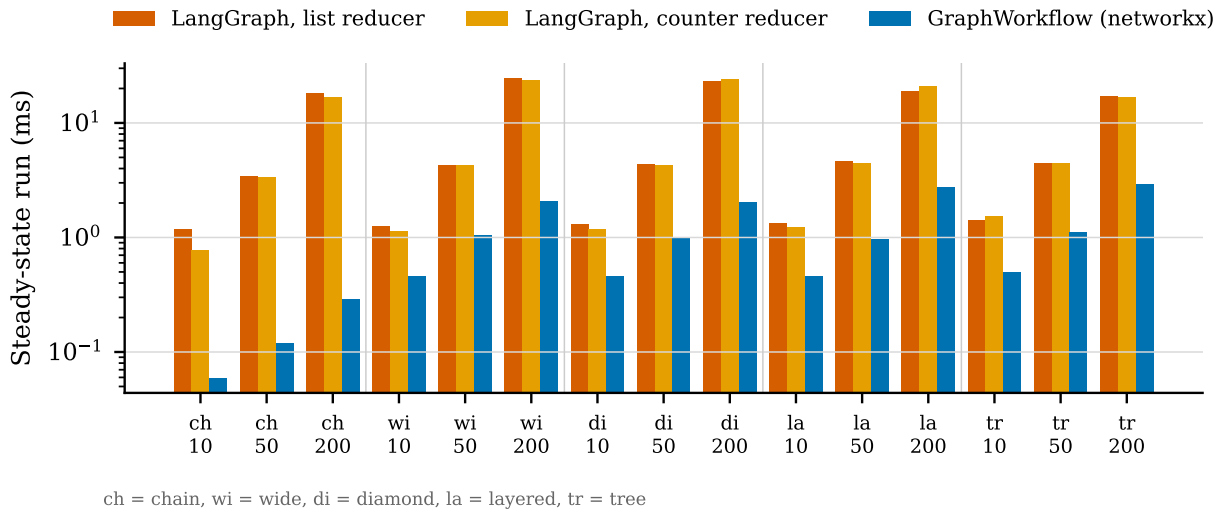


Figure 13: Steady-run cost of LangGraph under the appending list reducer (headline configuration) and an $O(1)$ counter reducer, against GraphWorkflow-networkx, log scale.

9 Discussion, Limitations, and Future Work

9.1 What the numbers do and do not claim

The evaluation measures orchestration overhead on static DAGs of no-op nodes. It does not measure end-to-end agent quality, token throughput, or behavior under real LLM latency, where a 500 ms node renders both frameworks' per-node overhead invisible in any single execution. The honest summary is that GraphWorkflow is one to two orders of magnitude cheaper at every measured phase of the graph lifecycle, with the gap widest exactly where overhead compounds: deep graphs, high execution rates, and repeated construction. What the measurements do not capture is feature coverage, which is the subject of the next subsection.

9.2 Feature asymmetry

Part of LangGraph's measured overhead purchases semantics GraphWorkflow does not offer: conditional edges, cycles with runtime termination conditions (GraphWorkflow's `max_loops` iterates the whole graph, a coarser construct), mid-graph interrupts for human approval, and pluggable superstep-granularity persistence. Workloads that need dynamic control flow inside the graph are outside GraphWorkflow's current model, and LangGraph remains the appropriate tool for them. The measured gaps are the price of generality imposed on workloads that do not use it, an instance of the classic pay-for-what-you-use argument.

9.3 Concurrency ceiling

The scheduler is thread-based and in-process. For I/O-bound agent calls this is the right ceiling on current CPython (the GIL is released during network waits [5]), but it caps CPU-bound node work in the manner of any serial-fraction limit [2] and offers no horizontal scale-out; the widest useful layer is bounded by ~95% of core count for dispatch and by provider rate limits in practice. The layer barrier also forfeits parallelism when node durations within a layer are skewed: a successor of a fast node waits for the slowest sibling (Section 3.3). An edge-triggered ready-set scheduler would recover this at the cost of per-node dependency counting on the hot path; measuring whether the recovered overlap ever exceeds the added bookkeeping under realistic LLM latency distributions is an open experiment.

9.4 Benchmark scope

Beyond the fairness limits of Section 6.4, three scope limits deserve emphasis. The suite tops out at 200 nodes; both systems' scaling curves are near-linear through that range, but neither memory behavior nor scheduler behavior at 10^4-node scale is characterized. Only the synchronous entry points are measured (`arun` currently delegates to a thread). And all numbers come from one machine, one Python version, and one OS; the harness records provenance precisely so that other environments can be added, and we consider cross-platform replication, peak-RSS, allocation counts, and CPU-utilization traces the most valuable missing measurements.

9.5 Future directions

Five directions follow naturally from the compile-once design. *Distributed execution*: the compiled plan is a plain layered list of picklable specifications, so partitioning layers across processes or machines, with the output map as the only shared state, is a natural extension, and multilevel partitioners [20] are the classical tool for minimizing cross-partition edges. *Heterogeneous and GPU-aware scheduling*: compile-time resource-class annotations plus per-class pools would extend the single-pool model without touching the sweep. *Compiler-assisted optimization*: the compile phase is the hook for fusing linear chains, batching sibling calls to one model endpoint, and precomputing prompt skeletons. *Dynamic rewriting*: compiling guard nodes into alternative sub-plans would close part of the expressiveness gap with LangGraph while preserving the static hot path. *Scale*: pushing the suite to 10^3 to 10^4 nodes, where the $O(N{+}M)$ compile bound predicts, but does not yet demonstrate, continued linearity.

10 Conclusion

Agent workflows are executed often enough, from cold enough starts, and at large enough fan-outs that orchestration overhead has become a systems problem rather than a rounding error. GraphWorkflow addresses it with an old idea applied cleanly: move every analysis a static DAG permits into an explicit, cached, invalidation-safe compile step, and leave the run loop nothing to do but sweep a frozen plan with a single shared pool and an inline path for sequential segments. A backend abstraction keeps the engine independent of any one graph library and confines the choice of library to the only phase where it can matter. On an open benchmark suite the design yields geometric-mean improvements over LangGraph of 7.0× in steady-state execution, 21.6× to 31.3× in compilation, and 7.9× on the cold path, with the gap growing to 62.5× exactly where the model predicts, on deep graphs where per-superstep machinery compounds. The same simplification shows up in the programming model: expressing the static-DAG common case requires no state schema, no reducers, no explicit entry and exit wiring, and no recursion configuration. The cost model and the released harness are intended to outlive the specific numbers: they give the agent-infrastructure community a way to compare orchestrators by regime rather than by anecdote.

References

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. Tensorflow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 265–283, 2016.

[2] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.

[3] Apache Software Foundation. Apache airflow: A platform to programmatically author, schedule and monitor workflows. `https://airflow.apache.org`, 2015.

[4] Yadu Babuji, Anna Woodard, Zhuozhao Li, Daniel S. Katz, Ben Clifford, Rohan Kumar, Lukasz Lacinski, Ryan Chard, Justin M. Wozniak, Ian Foster, Michael Wilde, and Kyle Chard. Parsl: Pervasive parallel programming in Python. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing (HPDC 19)*, pages 25–36, 2019.

[5] David Beazley. Understanding the Python GIL. PyCon US, 2010.

[6] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46 (5):720–748, 1999.

[7] Richard P. Brent. The parallel evaluation of general arithmetic expressions. *Journal of the ACM*, 21(2):201–206, 1974.

[8] Edward G. Coffman, Jr. and Ronald L. Graham. Optimal scheduling for two-processor systems. *Acta Informatica*, 1(3):200–213, 1972.

[9] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.

[10] Gábor Csárdi and Tamás Nepusz. The igraph software package for complex network research. *InterJournal*, Complex Systems: 1695, 2006.

[11] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[12] Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified data processing on large clusters. In *6th USENIX Symposium on Operating Systems Design and Implementation (OSDI 04)*, pages 137–150, 2004.

[13] Kye Gomez. Swarms: An enterprise-grade, production-ready multi-agent orchestration framework. `https://github.com/kyegomez/swarms`, 2023. Version 14.0.0.

[14] Ronald L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969.

[15] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, 2008.

[16] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for multi-agent collaborative framework. arXiv preprint arXiv:2308.00352, 2023.

[17] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: Distributed data-parallel programs from sequential building blocks. In *Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems (EuroSys 07)*, pages 59–72, 2007.

[18] Donald B. Johnson. Finding all the elementary circuits of a directed graph. *SIAM Journal on Computing*, 4(1):77–84, 1975.

[19] Arthur B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962.

[20] George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998.

[21] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP 23)*, pages 611–626, 2023.

[22] LangChain, Inc. LangGraph: A low-level orchestration framework for building stateful, multi-actor applications with LLMs. `https://github.com/langchain-ai/langgraph`, 2024. Version 1.0.4.

[23] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[24] Grzegorz Malewicz, Matthew H. Austern, Aart J.C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: A system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, pages 135–146, 2010.

[25] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 561–577, 2018.

[26] João Moura. CrewAI: A framework for orchestrating role-playing, autonomous AI agents. `https://github.com/crewAIInc/crewAI`, 2023.

[27] Derek G. Murray, Frank McSherry, Rebecca Isaacs, Michael Isard, Paul Barham, and Martín Abadi. Naiad: A timely dataflow system. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP 13)*, pages 439–455, 2013.

[28] Netflix, Inc. Metaflow: A framework for real-life data science and machine learning. `https://metaflow.org`, 2019.

[29] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST 23)*, pages 1–22, 2023.

[30] Prefect Technologies, Inc. Prefect: Modern workflow orchestration for data and ML engineers. `https://www.prefect.io`, 2018.

[31] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 15174–15186, 2024.

[32] Matthew Rocklin. Dask: Parallel computation with blocked algorithms and task scheduling. In *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, pages 126–132, 2015.

[33] Mohammad Shahrad, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 205–218, 2020.

[34] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[35] Spotify AB. Luigi: A Python module for building complex pipelines of batch jobs. `https://github.com/spotify/luigi`, 2012.

[36] Robert E. Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.

[37] Temporal Technologies, Inc. Temporal: Durable execution platform. `https://temporal.io`, 2019.

[38] Matthew Treinish, Ivan Carvalho, Georgios Tsilimigkounakis, and Nahum Sa. rustworkx: A high-performance graph library for Python. *Journal of Open Source Software*, 7(79):3968, 2022.

[39] Leslie G. Valiant. A bridging model for parallel computation. *Communications of the ACM*, 33(8):103–111, 1990.

[40] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. arXiv preprint arXiv:2406.04692, 2024.

[41] Liang Wang, Mengyuan Li, Yinqian Zhang, Thomas Ristenpart, and Michael Swift. Peeking behind the curtains of serverless platforms. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 133–146, 2018.

[42] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed H. Awadallah, Ryen W. White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155, 2023.

[43] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[44] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 15–28, 2012.

A Full Build and Compile Results

Table 4 completes the measurement grid of Table 2 with the build and compile phases (medians of 9 samples, milliseconds).

Table 4: Build and compile medians (ms) for all topologies and sizes.

		GraphWorkflow (networkx)		GraphWorkflow (rustworkx)		LangGraph	
Topology	N	build	compile	build	compile	build	compile
chain	10	0.178	0.077	0.186	0.066	0.384	0.940
	50	0.290	0.156	0.250	0.115	0.716	3.517
	200	0.679	0.439	0.477	0.264	2.077	12.569
wide	10	0.160	0.071	0.154	0.055	0.327	0.822
	50	0.263	0.139	0.230	0.087	0.714	3.346
	200	0.595	0.351	0.510	0.220	2.189	12.512
diamond	10	0.186	0.073	0.198	0.063	0.370	0.940
	50	0.304	0.150	0.279	0.101	0.783	3.824
	200	0.785	0.410	0.678	0.220	2.074	13.366
layered	10	0.165	0.061	0.149	0.056	0.321	0.942
	50	0.368	0.154	0.329	0.105	0.671	4.200
	200	1.204	0.489	0.975	0.285	2.125	15.980
tree	10	0.138	0.072	0.159	0.053	0.338	0.843
	50	0.247	0.128	0.219	0.092	0.711	3.281
	200	0.629	0.398	0.542	0.232	2.027	12.554

B Validation Pseudocode

C Reproduction Guide

All artifacts live in the Swarms repository (`https://github.com/kyegomez/swarms`) under `tests/benchmarks/graph_workflow_b`

```
# 1. Run the full benchmark suite (writes results/latest.json + latest.md)
python3 tests/benchmarks/graph_workflow_benchmarks/graph_workflow_bench.py

# 2. Reducer ablation (O(1) counter reducer for LangGraph fan-in state)
python3 tests/benchmarks/graph_workflow_benchmarks/graph_workflow_bench.py \
    --lg-reducer counter \
    --out .../results/langgraph_counter_reducer.json

# 3. Regenerate every figure, table, and statistic in this paper
python3 tests/benchmarks/graph_workflow_benchmarks/graphworkflow_paper/analysis/analyze.py

# 4. Build the paper
cd tests/benchmarks/graph_workflow_benchmarks/graphworkflow_paper
tectonic main.tex # or: latexmk -pdf main.tex
```

The analysis script consumes only the two result JSON files; every number in the text is emitted as a LaTeX macro by the same script (`tables/stats.tex`), so the paper cannot silently drift from the data. Statistical conventions: reported values are medians of $n = 9$ samples after 2 warmups; intervals are 95% confidence intervals on the mean using Student's t with 8 degrees of freedom; aggregate speedups are geometric means of per-cell median ratios. Raw samples for every cell, and full environment provenance, are preserved in the JSON files.

Algorithm 4 FastValidate: structural validation from adjacency maps

Require: nodes V, edges E, maps succ, pred, entry set S, exit set T
Ensure: error list $\mathcal{E}$, warning list $\mathcal{W}$

$\mathcal{E} \leftarrow [\,]$; $\mathcal{W} \leftarrow [\,]$
if $\exists v \in V$ with no executable **then** append to $\mathcal{E}$
end if
if $E = \emptyset$ **then** warn "no edges"
end if
$I \leftarrow \{v : \text{succ}[v] = \emptyset \wedge \text{pred}[v] = \emptyset\}$; **if** $I \neq \emptyset$ warn "isolated"
if not isDAG() **then** warn "cycles present" ▷ $O(N{+}M)$, never cycle enumeration
end if
$R \leftarrow$ MultiSourceBFS(S, succ); **if** $V \setminus R \neq \emptyset$ warn "unreachable"
$R' \leftarrow$ MultiSourceBFS(T, pred); **if** $V \setminus R' \neq \emptyset$ warn "dead ends"
return $(\mathcal{E}, \mathcal{W})$

procedure MultiSourceBFS(*sources*, *adj*)
    seen ← set(*sources*); *frontier* ← list(*sources*)
    while *frontier* ≠ ∅ **do**
        $v \leftarrow$ pop from *frontier*
        for all $w \in adj[v]$ **do**
            if $w \notin$ *seen* **then** add w to *seen*, push w
            end if
        end for
    end while
    return *seen*
end procedure